

Rate Intelligence Platform

Help Guide & Technical Documentation

Phase 1 — Frontend-Only Release

Rate Intelligence Platform — Help Guide

Enterprise hotel revenue management app for Properties, Rooms, and Rate Plans.

1. Overview

The Rate Intelligence Platform is a frontend-only, no-build React application for managing a hotel portfolio: Properties, their Rooms, and each room's Rate Plans, with role-based access control. It runs entirely in the browser — there is no backend server yet; all data lives in memory for the session.

2. Running the App

No `npm install` is required. Double-click `start.bat`, or run `powershell -ExecutionPolicy Bypass -File serve.ps1`, then open `http://localhost:5173/`. React, react-router-dom, and lucide-react load via an `importmap` in `index.html`; a service worker Babel-transforms JSX at request time, so nothing needs to be compiled ahead of time.

3. Architecture

- **Rendering:** React 18, routed with `react-router-dom` (HashRouter).
- **State:** `DataContext` holds all Properties/Rooms/Rate Plans/Master Data as an in-memory reducer, exposing CRUD actions that already mirror future REST calls.
- **Styling:** a plain CSS design system in `src/styles/`, token-driven, no CSS framework.
- **Auth:** `AuthContext` simulates login with two demo roles; permissions are derived flags from `src/lib/permissions.js`, consumed via `usePermissions()`.

4. Modules

Module	Path	Purpose
Properties	<code>src/pages/properties/</code>	List, create/edit form, and profile view for hotel properties.
Rooms	<code>src/pages/rooms/</code>	Room inventory per property — occupancy, bed configuration, amenities, features.
Rate Plans	<code>src/pages/ratePlans/</code>	Pricing plans per room, including meal plan, cancellation policy, taxes & fees, and pricing periods.
Settings	<code>src/pages/settings/</code>	General, Defaults, Appearance, and Integrations configuration.
Master Data	<code>src/mocks/masterData.js</code>	Shared lookup lists (Room Types, Amenities, Room Templates) with full CRUD.

5. Core Workflow

A typical session: create a **Property** → add one or more **Rooms** to it → attach one or more **Rate Plans** to each room, each with its own meal plan, cancellation policy, and taxes/fees. Every entity supports Create, Edit, Archive/Restore, Duplicate, and (role-permitting) permanent delete, plus bulk actions from the list views. All forms carry an unsaved-changes guard that prompts before discarding in-progress edits.

6. User Roles

Role	Login	Access
Super Admin	<code>admin@ratebuzz.com</code> / <code>Admin@123</code>	Full access to all properties, rooms, rate plans, and integrations.
Property Owner	<code>owner@ratebuzz.com</code> / <code>Owner@123</code>	Scoped to own properties; no Integrations view; cannot permanently delete rooms/rate plans.

7. Notable Features

- Tabbed Property Profile (Overview, Contact, Address, Operational, Rooms, Rate Plans, Notes, Settings).
- CSV export and a guided Import Wizard (parse → validate → preview).
- Breadcrumb navigation and global search across all three entities.
- Room and Rate Plan quick-fill templates for faster data entry.

8. Backend Roadmap Planned

The app is deliberately structured for a straightforward backend migration — swapping `DataContext`'s in-memory reducer for HTTP calls, with no frontend redesign required:

- **Database:** new SQL Server 2022 database, `RATEBUZZ`.
- **Backend:** .NET Core 8.0 (C#), ASP.NET MVC.
- **Views:** every `.jsx` page stays uncompiled today so it can be ported into MVC Views/View Components without reconciling against bundler output.
- **Auth:** `usePermissions()` is the single seam meant to be swapped for real ASP.NET Identity/JWT claims.